Categorizing Evolutionary Changes

Objective

Students will learn to identify and classify software changes into four maintenance categories, Corrective, Adaptive, Perfective, and Preventive, and understand why software systems must evolve continuously.

Background

Software systems invariably require ongoing maintenance and evolution. As users' needs and environments change, software must be updated or it will "become progressively less useful"[1]. Indeed, one study notes that maintenance can consume about 60–70% of total software lifecycle effort[2]. According to IEEE/ISO definitions, maintenance includes any modifications after release to keep a system functioning in a changing environment[3].

The four main maintenance categories are:

- Corrective – fixing faults or defects. This includes addressing bugs or errors in design, logic, or code (often reported by users) to restore proper functionality[4].
- Adaptive – modifying software to work in a new environment. For example, changes to support a new operating system, hardware platform, or updated external interfaces (e.g., third-party services) fall under adaptive maintenance[5].
- Perfective – enhancing or refining features. This covers improvements based on evolving user requirements or feedback, such as adding new capabilities, improving performance, or removing obsolete features[6].
- Preventive – proactive updates to avert future problems. Examples include code refactoring, optimization, or updating documentation to prevent deterioration and ensure long-term stability[7].

These categories are not mutually exclusive in practice, but they help teams reason about why a change is being made. As Lehman's *Law of Continuing Change* states, "A program that is used in a real-world environment must necessarily change, or else become progressively less useful"[1]. This law underscores that maintenance and evolution are inherent to software's lifecycle. Regular maintenance (corrective, adaptive, perfective, and preventive) keeps software reliable and relevant in the face of changing requirements and technology.

Instructions

1. Read the 10 short change scenarios listed below. For each scenario, decide which maintenance category it represents: corrective, adaptive, perfective, or preventive.
2. In the team worksheet table (below), enter each scenario, your chosen category, and a brief justification explaining why you placed it in that category. Use course concepts and examples to support your reasoning.
3. Next, think of a real-world example for each scenario type. For instance, recall any software you've used or read about that underwent a similar change. Enter a concrete example (software name and context) in the last column.
4. Discuss as a team until you reach consensus on each classification and example. Try to cover all four categories among the scenarios. If you finish early, review and refine your justifications.
5. At the end of the activity, be prepared to briefly explain one of your classifications and examples to the class.

Mini Case List

For each item below, the team should determine the change category.

1. Login bug fix: A recent update corrects a fault where certain users cannot log in due to a coding error in the authentication module.
2. OS upgrade support: After users upgrade to a new version of the operating system, the software needs a patch to maintain compatibility.
3. New feature addition: Users request a “dark mode” feature for better usability at night, and developers add this feature in a new release.
4. Performance optimization: The development team refactors the image-processing algorithm to run faster and use less memory, anticipating increased data loads.
5. Regulatory compliance: A change to the software is needed to comply with a new data privacy law (e.g., adding data encryption and audit logging).
6. Feature removal: A rarely used reporting module is removed entirely because it is outdated and complicates maintenance.
7. Code cleanup: Developers reorganize and document code to improve maintainability, even though it does not change any user-visible functionality.
8. Third-party API change: An external service the app depends on has updated its API; the team modifies the integration code so the app continues to work.
9. User interface tweak: Based on user feedback, the layout of the dashboard is improved (buttons are rearranged and labels clarified) without adding new features.
10. Planned capacity increase: Anticipating more users, the team implements a database sharding strategy before any current performance issues arise.

Team Worksheet Section

Use the table below to record your work. Fill in each row with the scenario, the category you choose, your justification, and a real-world example. (You can copy/paste the row format to add more scenarios if needed.)

Scenario	Category	Justification (Why this category?)	Real-World Example (Software & Context)
1. Login bug fix			
2. OS upgrade support			
3. New feature addition			
4. Performance optimization			
5. Regulatory compliance			
6. Feature removal			
7. Code cleanup			
8. Third-party API change			
9. User interface tweak			

10. Planned capacity increase

---

[1] utc.edu

https://www.utc.edu/document/72156

[2] [3] Federal Information Processing Standards Publication: guideline on software maintenance

https://nvlpubs.nist.gov/nistpubs/Legacy/FIPS/fipspub106.pdf

[4] [5] [6] [7] The 4 Types Of Software Maintenance & How They Help | CAST Software

https://www.castsoftware.com/glossary/four-types-of-software-maintenance-how-they-help-your-organization-preventive-perfective-adaptive-corrective